Introduction

Crystal.Avalonia is a lightweight MVVM framework designed specifically for Avalonia UI applications. It provides a clean architecture pattern that makes building complex, cross-platform applications simpler and more maintainable.

Why Crystal.Avalonia?

When building large-scale Avalonia applications, you often face challenges like:

- **Code organization** - How to keep your code maintainable as the app grows
- **View/ViewModel binding** - Managing the relationship between views and their view models
- **Modularity** - How to split your app into independent, testable units
- **Dependency Injection** - Properly wiring up services across your application

Crystal.Avalonia addresses all of these concerns out of the box.

Core Concepts

1. CrystalApplication

The base application class that extends Avalonia's `Application` with modular development support:

```
public class MyApp : CrystalApplication
{
    public override void RegisterModules(IModuleRegistrar moduleRegistrar) { }
    public override void RegisterServices(IServiceCollection services) { }
    public override void CreateShell(IServiceProvider serviceProvider) { }
}
```

2. Module System

Modules are self-contained units of functionality that manage their own services and views:

```
public class MyModule : IModule
{
    public void RegisterServices(IServiceCollection services)
    {
        // Register module services here
    }

    public void InitializeModule(IServiceProvider serviceProvider)
```

```
    {
        // Initialize module here
    }
}
```

3. MVVM Binding

Crystal.Avalonia provides simple yet powerful View/ViewModel binding:

```
// Register the mapping
services.AddMvvmBindingTransient<MainView, MainViewModel>();

// In XAML, enable auto-binding
<Window xmlns:vm="using:Crystal.Avalonia"
        vm:ViewModelLocator.AutoWireViewModel="True">
```

Architecture Overview

```
┌──────────────────────────────────────────────────────┐
│                   CrystalApplication                 │
├──────────────────────────────────────────────────────┤
│  RegisterModules()    - Register IModule impls       │
│  RegisterServices()   - Register DI services         │
│  CreateShell()        - Create main window/view      │
└──────────────────────────────────────────────────────┘
                              │
          ┌───────────────────┼───────────────────┐
          ▼                   ▼                   ▼
┌────────────────┐    ┌────────────────┐    ┌────────────────┐
│   Module A     │    │   Module B     │    │  Main App      │
├────────────────┤    ├────────────────┤    ├────────────────┤
│ • Services     │    │ • Services     │    │ • ViewModels   │
│ • Views        │    │ • Views        │    │ • Views        │
│ • ViewModels   │    │ • ViewModels   │    │ • Services     │
└────────────────┘    └────────────────┘    └────────────────┘
```

Key Features

Feature	Description
Modular Architecture	Organize code into independent modules with their own services and views

Feature	Description
Automatic DI Wiring	ViewModels and Views are automatically registered and resolved
Flexible Binding	Two binding modes: `AddMvvmBindingTransient` and `AddMvvmBindingSingleton`
AOT Ready	Full support for trimming and AOT compilation
Cross-Platform	Supports all Avalonia platforms including WASM

Next Steps

- Getting Started - Create your first Crystal.Avalonia application
- API Reference - Explore the full API documentation

Getting Started

This tutorial will guide you through creating your first Crystal.Avalonia application.

Prerequisites

- .NET 10.0 SDK or later
- An IDE with Avalonia support (Visual Studio 2022+, Rider, or VS Code with Avalonia extension)

Step 1: Install the Template

First, install the Crystal.Avalonia project template:

```
dotnet new install Crystal.Avalonia.Template
```

This adds the `crystal.avalonia` template to `dotnet new`.

Step 2: Create a New Project

Create a new project using the template:

```
dotnet new crystal.avalonia -o MyFirstApp
cd MyFirstApp
```

The template creates a complete project structure:

```
MyFirstApp/
├── ModuleA/              # Example module
│   ├── ModuleA.cs
│   ├── ModuleAView.axaml
│   ├── ModuleAView.axaml.cs
│   ├── ModuleAViewModel.cs
│   └── ModuleA.csproj
├── ModuleB/              # Another example module
├── ShareBaseClass/       # Shared contracts
├── MyFirstApp/           # Main application
│   ├── App.axaml
│   ├── App.axaml.cs
│   ├── ViewModels/
│   │   └── MainViewModel.cs
│   └── Views/
│       ├── MainWindow.axaml
```

```
│       └── MainWindow.axaml.cs
├── MyFirstApp.Desktop/   # Desktop-specific code
└── MyFirstApp.csproj     # Main project file
```

Step 3: Explore the Project Structure

App.axaml.cs - Application Entry Point

```
public partial class App : CrystalApplication
{
    public override void Initialize()
    {
        AvaloniaXamlLoader.Load(this);
    }

    // Register modules (optional)
    public override void RegisterModules(IModuleRegistrar moduleRegistrar)
    {
        moduleRegistrar.RegisterModule<ModuleA>();
    }

    // Register View/ViewModel pairs
    public override void RegisterServices(IServiceCollection services)
    {
        services.AddMvvmBindingTransient<MainView, MainViewModel>();
    }

    // Create the main window
    public override void CreateShell(IServiceProvider serviceProvider)
    {
        if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime desktop)
            desktop.MainWindow = new MainWindow();
    }
}
```

MainViewModel.cs - ViewModel Example

```
public class MainViewModel : ObservableObject
{
    private string _greeting = "Welcome to Crystal.Avalonia!";

    public string Greeting
    {
        get => _greeting;
```

```
        set => SetProperty(ref _greeting, value);
    }
}
```

MainView.axaml - View with Data Binding

```
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:vm="using:Crystal.Avalonia"
             vm:ViewModelLocator.AutoWireViewModel="True">

    <StackPanel Margin="20">
        <TextBlock Text="{Binding Greeting}"
                   FontSize="24"
                   HorizontalAlignment="Center"/>
    </StackPanel>
</UserControl>
```

Step 4: Create a Module

Modules are independent units that can be shared across projects.

1. Define the Module

```
using Crystal.Avalonia;
using Microsoft.Extensions.DependencyInjection;

namespace MyApp.Modules.Settings;

public class SettingsModule : IModule
{
    public void RegisterServices(IServiceCollection services)
    {
        // Register View/ViewModel pair for this module
        services.AddMvvmBindingTransient<SettingsView, SettingsViewModel>();
    }

    public void InitializeModule(IServiceProvider serviceProvider)
    {
        // Perform initialization (e.g., load settings, subscribe to events)
    }
}
```

2. Register the Module

In `App.axaml.cs`:

```
public override void RegisterModules(IModuleRegistrar moduleRegistrar)
{
    moduleRegistrar.RegisterModule<SettingsModule>();
}
```

Step 5: Choose Binding Mode

Crystal.Avalonia provides two binding modes:

Transient (Default)

Both View and ViewModel are created each time:

```
services.AddMvvmBindingTransient<MainView, MainViewModel>();
// Use this when: Each view instance needs its own ViewModel instance
```

Singleton

View is transient, but ViewModel is singleton:

```
services.AddMvvmBindingSingleton<SettingsView, SettingsViewModel>();
// Use this when: The same ViewModel instance should be shared
```

Step 6: Run the Application

```
dotnet run
```

For desktop applications:

```
dotnet run --project MyFirstApp.Desktop
```

Understanding the MVVM Flow

1. **Startup** - `CrystalApplication.OnFrameworkInitializationCompleted()` is called
2. **Module Registration** - All modules are registered via `RegisterModules()`
3. **Service Registration** - Services are registered via `RegisterServices()` and each module's `RegisterServices()`
4. **Shell Creation** - `CreateShell()` is called with the fully configured `ServiceProvider`

5. **View Resolution** - When a View with `AutoWireViewModel="True"` loads, its corresponding ViewModel is resolved from DI and assigned to `DataContext`

Next Steps

- Create a More Complex App - Build a complete application step by step
- Module Development - Deep dive into the module system
- API Reference - Explore all available APIs

Tutorial: Create Your First Application

In this tutorial, we'll build a complete counter application with Crystal.Avalonia to understand the framework better.

What We're Building

A simple counter application with:

- Increment/Decrement buttons
- Current count display
- Reset functionality

Step 1: Set Up the Project

```
dotnet new crystal.avalonia -o CounterApp
cd CounterApp
```

Step 2: Define the ViewModel

Create `CounterViewModel.cs`:

```
using System;

public class CounterViewModel : ObservableObject
{
    private int _count;

    public int Count
    {
        get => _count;
        set => SetProperty(ref _count, value);
    }

    public void Increment() => Count++;
    public void Decrement() => Count--;
    public void Reset() => Count = 0;
}
```

Step 3: Create the View

Create `CounterView.axaml`:

```xml
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:vm="using:Crystal.Avalonia"
             vm:ViewModelLocator.AutoWireViewModel="True">

    <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center" Spacing="10">
        <TextBlock Text="{Binding Count}"
                   FontSize="48"
                   HorizontalAlignment="Center"/>

        <StackPanel Orientation="Horizontal" Spacing="10">
            <Button Content="-" Command="{Binding Decrement}" Width="50"/>
            <Button Content="Reset" Command="{Binding Reset}"/>
            <Button Content="+" Command="{Binding Increment}" Width="50"/>
        </StackPanel>
    </StackPanel>
</UserControl>
```

Create `CounterView.axaml.cs`:

```csharp
using Avalonia.Controls;

namespace CounterApp.Views;

public partial class CounterView : UserControl
{
    public CounterView()
    {
        InitializeComponent();
    }
}
```

Step 4: Register in App.axaml.cs

```csharp
public override void RegisterServices(IServiceCollection services)
{
    services.AddMvvmBindingTransient<CounterView, CounterViewModel>();
}
```

Step 5: Navigate to the Counter

In your `MainWindow.axaml`, add a button to navigate:

```
<StackPanel>
    <Button Content="Open Counter"
            Click="OpenCounter"/>

    <ContentControl Name="ContentArea"/>
</StackPanel>
```

In `MainWindow.axaml.cs`:

```
public void OpenCounter()
{
    var counterView = MvvmManager.ServiceProvider!.GetService<CounterView>();
    ContentArea.Content = counterView;
}
```

Step 6: Run

```
dotnet run
```

Key Concepts Demonstrated

Concept	How It's Used
`ObservableObject`	ViewModel inherits from it to support property change notifications
`SetProperty()`	Used in setters to automatically notify UI of changes
`Command` binding	Buttons bound directly to ViewModel methods
`AutoWireViewModel`	XAML attribute that triggers automatic DataContext injection
`MvvmManager.ServiceProvider`	Access to DI container for manual view resolution

Complete Code

ViewModel

```
using System;

namespace CounterApp.ViewModels;
```

```
public class CounterViewModel : ObservableObject
{
    private int _count;

    public int Count
    {
        get => _count;
        set => SetProperty(ref _count, value);
    }

    public void Increment() => Count++;
    public void Decrement() => Count--;
    public void Reset() => Count = 0;
}
```

View (AXAML)

```
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:vm="using:Crystal.Avalonia"
             vm:ViewModelLocator.AutoWireViewModel="True">

    <StackPanel HorizontalAlignment="Center"
                VerticalAlignment="Center"
                Spacing="10">

        <TextBlock Text="{Binding Count}"
                   FontSize="48"
                   HorizontalAlignment="Center"/>

        <StackPanel Orientation="Horizontal" Spacing="10">
            <Button Content="-" Command="{Binding Decrement}" Width="50"/>
            <Button Content="Reset" Command="{Binding Reset}"/>
            <Button Content="+" Command="{Binding Increment}" Width="50"/>
        </StackPanel>
    </StackPanel>
</UserControl>
```

Next Steps

- Module Development - Learn how to create reusable modules
- Dependency Injection - Advanced DI patterns

Tutorial: Module Development

Modules are the core building blocks of Crystal.Avalonia applications. They provide a way to organize your code into independent, reusable units.

What is a Module?

A module is a class that implements `IModule` and encapsulates:

- **Services** - Dependencies that the module provides
- **Views** - UI components specific to the module
- **ViewModels** - Business logic for the module's views
- **Initialization Logic** - Setup that runs when the app starts

Creating a Module

Step 1: Define the Module Class

```
using Crystal.Avalonia;
using Microsoft.Extensions.DependencyInjection;
using System;

namespace MyApp.Modules.UserManagement;

public class UserManagementModule : IModule
{
    public void RegisterServices(IServiceCollection services)
    {
        // Register View/ViewModel pairs
        services.AddMvvmBindingTransient<UserListView, UserListViewModel>();
        services.AddMvvmBindingTransient<UserDetailView, UserDetailViewModel>();

        // Register other services
        services.AddSingleton<IUserRepository, UserRepository>();
        services.AddTransient<UserService>();
    }

    public void InitializeModule(IServiceProvider serviceProvider)
    {
        // Perform initialization tasks
        var userService = serviceProvider.GetRequiredService<UserService>();
        userService.LoadUsers();
    }
}
```

Step 2: Register the Module

In your `App.axaml.cs`:

```
public override void RegisterModules(IModuleRegistrar moduleRegistrar)
{
    moduleRegistrar.RegisterModule<UserManagementModule>();
}
```

Module Project Structure

For larger applications, consider placing each module in its own project:

```
MySolution/
├── MyApp/                    # Main application
│   └── App.axaml.cs
├── MyApp.Modules.UserManagement/  # Module in separate project
│   ├── UserManagementModule.cs
│   ├── Views/
│   ├── ViewModels/
│   └── Services/
└── MyApp.Modules.Settings/   # Another module
    └── ...
```

Sharing Modules Between Apps

Modules can be easily shared across different applications:

1. Create a separate project for the module
2. Add the module project as a reference to your main app
3. Register the module in your app

Module Project File (.csproj)

```
<Project Sdk="Microsoft.NET.Sdk">

    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
    </PropertyGroup>

    <ItemGroup>
        <PackageReference Include="Crystal.Avalonia" />
    </ItemGroup>
```

```
</Project>
```

Consuming in Main App

```
// In App.axaml.cs of the main application
public override void RegisterModules(IModuleRegistrar moduleRegistrar)
{
    // Register modules from referenced projects
    moduleRegistrar.RegisterModule<MyApp.Modules.UserManagement.UserManagementModule>();
    moduleRegistrar.RegisterModule<MyApp.Modules.Settings.SettingsModule>();
}
```

Advanced Module Patterns

Module with Dependencies

```
public class ReportingModule : IModule
{
    public void RegisterServices(IServiceCollection services)
    {
        services.AddMvvmBindingTransient<ReportView, ReportViewModel>();
    }

    public void InitializeModule(IServiceProvider serviceProvider)
    {
        // Get services from the main app
        var logger = serviceProvider.GetRequiredService<ILogger>();
        logger.LogInformation("Reporting module initialized");
    }
}
```

Conditional Module Loading

```
public override void RegisterModules(IModuleRegistrar moduleRegistrar)
{
    // Only load admin module for admin users
    if (User.IsAdmin)
    {
        moduleRegistrar.RegisterModule<AdminModule>();
    }
```

```
    moduleRegistrar.RegisterModule<CommonModule>();
}
```

Module with Shared Base Class

```
// Base class for all modules in your application
public abstract class AppModule : IModule
{
    protected IServiceCollection? Services { get; private set; }
    protected IServiceProvider? ServiceProvider { get; private set; }

    public void RegisterServices(IServiceCollection services)
    {
        Services = services;
        OnRegisterServices();
    }

    public void InitializeModule(IServiceProvider serviceProvider)
    {
        ServiceProvider = serviceProvider;
        OnInitialize();
    }

    protected abstract void OnRegisterServices();
    protected abstract void OnInitialize();
}

// Usage
public class MyModule : AppModule
{
    protected override void OnRegisterServices()
    {
        Services!.AddMvvmBindingTransient<MyView, MyViewModel>();
    }

    protected override void OnInitialize()
    {
        var config = ServiceProvider!.GetRequiredService<AppConfig>();
        // Use config...
    }
}
```

Best Practices

Practice	Why
Keep modules focused	Each module should represent a single feature area
Define module contracts	Use interfaces for cross-module communication
Minimize module dependencies	Modules should be as independent as possible
Use shared base classes	Reduces boilerplate for large module collections
Consider lazy loading	For large modules that aren't always needed

Cross-Module Communication

Option 1: Shared Services

```
// In a shared project
public interface INavigationService
{
    void NavigateTo<TView>() where TView : Control;
}

// In ModuleA
public class NavigationService : INavigationService { ... }

// In ModuleA's RegisterServices
services.AddSingleton<INavigationService, NavigationService>();

// In ModuleB, use the shared service
public class ModuleB : IModule
{
    public void InitializeModule(IServiceProvider sp)
    {
        var nav = sp.GetRequiredService<INavigationService>();
        nav.NavigateTo<ModuleAView>();
    }
}
```

Option 2: Event Aggregator

```
// Simple event aggregator
public interface IEventAggregator
{
    void Subscribe<TEvent>(Action<TEvent> handler);
```

```
    void Publish<TEvent>(TEvent event);
}

// Usage
public class ModuleA : IModule
{
    public void InitializeModule(IServiceProvider sp)
    {
        var ea = sp.GetRequiredService<IEventAggregator>();
        ea.Subscribe<UserCreatedEvent>(e => /* react */);
    }
}

public class ModuleB : IModule
{
    public void InitializeModule(IServiceProvider sp)
    {
        var ea = sp.GetRequiredService<IEventAggregator>();
        ea.Publish(new UserCreatedEvent { UserId = 123 });
    }
}
```

Next Steps

- Dependency Injection - Advanced DI patterns and service registration
- Navigation - Implementing navigation between views

Tutorial: MVVM Pattern in Crystal.Avalonia

This tutorial explains how the MVVM (Model-View-ViewModel) pattern works in Crystal.Avalonia.

MVVM Overview

```
┌──────────────┐          ┌──────────────────┐          ┌──────────────┐
│    View      │◄─────────│   ViewModel      │──────────│    Model     │
│  (AXAML)     │  Data    │   (C# Class)     │  Data    │  (C# Class)  │
└──────────────┘  Binding └──────────────────┘  Access  └──────────────┘
```

- **Model** - Your data and business logic
- **ViewModel** - Exposes data and commands for the View
- **View** - Declarative UI (AXAML)

ViewModel Base Class

Crystal.Avalonia uses `ObservableObject` as the base class for ViewModels. It provides:

- `SetProperty()` - For notifying the UI when properties change
- `OnPropertyChanged()` - For manual property change notifications

Example: Basic ViewModel

```
using System;

public class MainViewModel : ObservableObject
{
    private string _title = "My App";
    private int _itemCount;
    private bool _isLoading;

    public string Title
    {
        get => _title;
        set => SetProperty(ref _title, value);
    }

    public int ItemCount
    {
        get => _itemCount;
        set => SetProperty(ref _itemCount, value);
    }
```

```
    public bool IsLoading
    {
        get => _isLoading;
        set => SetProperty(ref _isLoading, value);
    }
}
```

Property Binding

In ViewModel

```
private string _message;
public string Message
{
    get => _message;
    set => SetProperty(ref _message, value);  // Notifies UI of change
}
```

In View (AXAML)

```
<TextBlock Text="{Binding Message}"/>
```

When `Message` is updated in the ViewModel, the TextBlock automatically updates.

Command Binding

Commands allow the View to invoke methods in the ViewModel.

ViewModel with Commands

```
using System.Windows.Input;

public class CounterViewModel : ObservableObject
{
    private int _count;

    public int Count
    {
        get => _count;
        set => SetProperty(ref _count, value);
    }

    // Commands using Action
```

```
    public void Increment()
    {
        Count++;
    }

    public void Decrement()
    {
        Count--;
    }

    public void Reset()
    {
        Count = 0;
    }
}
```

Command Binding in View

```
<StackPanel Orientation="Horizontal" Spacing="10">
    <Button Content="-" Command="{Binding Decrement}"/>
    <Button Content="Reset" Command="{Binding Reset}"/>
    <Button Content="+" Command="{Binding Increment}"/>
</StackPanel>
```

Collection Binding

ViewModel with ObservableCollection

```
using System.Collections.ObjectModel;

public class ListViewModel : ObservableObject
{
    public ObservableCollection<ItemViewModel> Items { get; } = new();

    public ListViewModel()
    {
        // Initialize with some items
        Items.Add(new ItemViewModel { Name = "Item 1" });
        Items.Add(new ItemViewModel { Name = "Item 2" });
    }
}

public class ItemViewModel : ObservableObject
{
```

```
    private string _name;
    private bool _isSelected;

    public string Name
    {
        get => _name;
        set => SetProperty(ref _name, value);
    }

    public bool IsSelected
    {
        get => _isSelected;
        set => SetProperty(ref _isSelected, value);
    }
}
```

List Binding in View

```
<ListBox ItemsSource="{Binding Items}">
    <ListBox.ItemTemplate>
        <DataTemplate>
            <StackPanel Orientation="Horizontal">
                <CheckBox IsChecked="{Binding IsSelected}"/>
                <TextBlock Text="{Binding Name}"/>
            </StackPanel>
        </DataTemplate>
    </ListBox.ItemTemplate>
</ListBox>
```

Automatic ViewModel Binding

Crystal.Avalonia can automatically inject ViewModels into Views.

Enable Auto-Wiring

```
<UserControl xmlns:vm="using:Crystal.Avalonia"
             vm:ViewModelLocator.AutoWireViewModel="True">
```

How It Works

1. View loads with `AutoWireViewModel="True"`
2. Crystal looks up the registered ViewModel type for this View
3. Resolves the ViewModel from the DI container
4. Assigns it to `DataContext`

Manual Resolution

You can also manually resolve ViewModels:

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        Loaded += OnLoaded;
    }

    private void OnLoaded(object sender, RoutedEventArgs e)
    {
        var vm = MvvmManager.ServiceProvider!.GetService<MainViewModel>();
        DataContext = vm;
    }
}
```

Two-Way Binding

By default, bindings are one-way. Use two-way for input controls:

```
<TextBox Text="{Binding UserName, Mode=TwoWay}"/>
<TextBox Text="{Binding SearchQuery, Mode=TwoWay}"/>
```

Best Practices

Practice	Description
Keep ViewModels thin	Move logic to services/models
Use interfaces	Makes testing easier
Avoid code-behind	Keep UI logic in ViewModel
Use ObservableCollection	For collections that change
Immutable models	Use immutable records for data transfer

Complete Example

ViewModel

```
public class OrderViewModel : ObservableObject
{
    private string _orderId = "";
    private OrderStatus _status;
    private bool _isEditing;

    public string OrderId
    {
        get => _orderId;
        set => SetProperty(ref _orderId, value);
    }

    public OrderStatus Status
    {
        get => _status;
        set => SetProperty(ref _status, value);
    }

    public bool IsEditing
    {
        get => _isEditing;
        set => SetProperty(ref _isEditing, value);
    }

    public void Approve() => Status = OrderStatus.Approved;
    public void Reject() => Status = OrderStatus.Rejected;
    public void Edit() => IsEditing = true;
    public void Save() => IsEditing = false;
}

public enum OrderStatus { Pending, Approved, Rejected }
```

View

```
<UserControl xmlns:vm="using:Crystal.Avalonia"
             xmlns:sys="clr-namespace:System;assembly=mscorlib"
             vm:ViewModelLocator.AutoWireViewModel="True">

    <StackPanel Margin="20" Spacing="15">
        <StackPanel Orientation="Horizontal" Spacing="10">
            <TextBlock Text="Order ID:" VerticalAlignment="Center"/>
            <TextBlock Text="{Binding OrderId}"
                       FontWeight="Bold"
                       VerticalAlignment="Center"/>
        </StackPanel>
```

```
        <StackPanel Orientation="Horizontal" Spacing="10">
            <TextBlock Text="Status:" VerticalAlignment="Center"/>
            <TextBlock Text="{Binding Status}"
                       VerticalAlignment="Center"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal" Spacing="10">
            <Button Content="Approve"
                    Command="{Binding Approve}"
                    IsVisible="{Binding !IsEditing}"/>
            <Button Content="Reject"
                    Command="{Binding Reject}"
                    IsVisible="{Binding !IsEditing}"/>
            <Button Content="Edit"
                    Command="{Binding Edit}"
                    IsVisible="{Binding !IsEditing}"/>
            <Button Content="Save"
                    Command="{Binding Save}"
                    IsVisible="{Binding IsEditing}"/>
        </StackPanel>
    </StackPanel>
</UserControl>
```

Next Steps

- Dependency Injection - Learn about DI in Crystal.Avalonia
- Module Development - Organize code into modules

Tutorial: Dependency Injection

Crystal.Avalonia uses Microsoft.Extensions.DependencyInjection for dependency injection. This tutorial covers DI patterns and best practices.

Basic Concepts

Constructor Injection (Recommended)

```
public class UserListViewModel : ObservableObject
{
    private readonly IUserService _userService;
    private readonly ILogger _logger;

    // Dependencies are injected through the constructor
    public UserListViewModel(IUserService userService, ILogger logger)
    {
        _userService = userService;
        _logger = logger;
    }
}
```

Property Injection

```
public class SettingsViewModel : ObservableObject
{
    public IConfigService? ConfigService { get; set; }
}
```

Registering Services

In Module.RegisterServices

```
public class MyModule : IModule
{
    public void RegisterServices(IServiceCollection services)
    {
        // Transient - New instance each time
        services.AddTransient<IUserService, UserService>();

        // Singleton - Same instance shared
        services.AddSingleton<ISettingsService, SettingsService>();

        // Scoped - One instance per scope
```

```
        services.AddScoped<IDbContext, AppDbContext>();

        // Instance - Pre-created instance
        services.AddInstance(new AppMetrics());
    }
}
```

In App.RegisterServices

```
public override void RegisterServices(IServiceCollection services)
{
    // Register application-wide services
    services.AddSingleton<IAppConfiguration, AppConfiguration>();
    services.AddSingleton<INavigationService, NavigationService>();

    // Register View/ViewModel pairs
    services.AddMvvmBindingTransient<MainView, MainViewModel>();
}
```

Service Lifetimes

Lifetime	When to Use	Behavior
`Transient`	Lightweight, stateless services	New instance each time
`Singleton`	Shared state, configuration	Same instance always
`Scoped`	Per-operation data (e.g., DB context)	One instance per scope

Example: Choosing the Right Lifetime

```
public void RegisterServices(IServiceCollection services)
{
    // Transient: New instance for each request
    // Good for: ViewModels, lightweight services
    services.AddTransient<MainViewModel>();
    services.AddTransient<IIdGenerator, GuidIdGenerator>();

    // Singleton: Shared across the app
    // Good for: Configuration, logging, caching
    services.AddSingleton<AppSettings>();
    services.AddSingleton<ILogger, FileLogger>();

    // Scoped: Per operation/scope
```

```
    // Good for: Database contexts
    services.AddScoped<IDbContext>(sp => new MyDbContext());
}
```

Advanced Patterns

Factory Pattern

```
public interface IViewModelFactory
{
    TViewModel Create<TViewModel>() where TViewModel : class;
}

public class ViewModelFactory : IViewModelFactory
{
    private readonly IServiceProvider _serviceProvider;

    public ViewModelFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public TViewModel Create<TViewModel>() where TViewModel : class
    {
        return _serviceProvider.GetRequiredService<TViewModel>();
    }
}

// Registration
services.AddSingleton<IViewModelFactory, ViewModelFactory>();

// Usage
public class DetailViewModel : ObservableObject
{
    private readonly IViewModelFactory _factory;

    public DetailViewModel(IViewModelFactory factory)
    {
        _factory = factory;
    }

    public void OpenItem(ItemViewModel item)
    {
        var vm = _factory.Create<ItemDetailViewModel>();
        vm.Load(item);
```

```
    }
}
```

Lazy Resolution

```
public class MyViewModel : ObservableObject
{
    private readonly Lazy<IHeavyService> _heavyService;

    public MyViewModel(Lazy<IHeavyService> heavyService)
    {
        _heavyService = heavyService;
    }

    public void DoWork()
    {
        // Service is only created when accessed
        var service = _heavyService.Value;
        service.Execute();
    }
}
```

Options Pattern

```
// Configuration
public class AppSettings
{
    public string ApiBaseUrl { get; set; } = "";
    public int TimeoutSeconds { get; set; } = 30;
}

// Registration
var settings = new AppSettings
{
    ApiBaseUrl = "https://api.example.com",
    TimeoutSeconds = 60
};
services.AddSingleton(settings);

// Usage
public class ApiService : IApiService
{
    public ApiService(AppSettings settings)
    {
```

```
        var baseUrl = settings.ApiBaseUrl;
        var timeout = settings.TimeoutSeconds;
    }
}
```

Cross-Module Service Sharing

Define Service in Main App

```
// Main App
public override void RegisterServices(IServiceCollection services)
{
    services.AddSingleton<INavigationService, NavigationService>();
}
```

Use in Modules

```
public class MyModule : IModule
{
    public void InitializeModule(IServiceProvider serviceProvider)
    {
        // NavigationService was registered in the main app
        var nav = serviceProvider.GetRequiredService<INavigationService>();
        nav.NavigateToHome();
    }
}
```

Testing with DI

Register Test Doubles

```
public void Setup()
{
    var services = new ServiceCollection();

    // Register test doubles
    services.AddSingleton<IUserService, MockUserService>();
    services.AddSingleton<ILogger, MockLogger>();

    // Register the ViewModel
    services.AddTransient<UserListViewModel>();

    var provider = services.BuildServiceProvider();
```

```
    var vm = provider.GetRequiredService<UserListViewModel>();
}
```

Testing ViewModels

```
public class UserListViewModelTests
{
    [Fact]
    public void LoadUsers_UpdatesItemsList()
    {
        // Arrange
        var mockService = new MockUserService();
        var vm = new UserListViewModel(mockService);

        // Act
        vm.RefreshCommand.Execute();

        // Assert
        Assert.NotEmpty(vm.Users);
    }
}
```

Common Pitfalls

1. Captive Dependency

```
// WRONG: DbContext captured by singleton
services.AddSingleton<IService>(sp =>
{
    var dbContext = sp.GetRequiredService<DbContext>(); // ❌
    return new Service(dbContext);
});

// CORRECT: Scoped DbContext
services.AddScoped<MyService>(sp =>
{
    var dbContext = sp.GetRequiredService<DbContext>(); // ✓
    return new Service(dbContext);
});
```

2. Circular Dependencies

```
// WRONG: Circular dependency
public class A
{
    public A(B b) { }
}
public class B
{
    public B(A a) { }  // ❌ Circular!

// CORRECT: Break the cycle with an interface
public interface IA { }
public interface IB { }
public class A : IA
{
    public A(IB b) { }
}
```

Next Steps

- Navigation - Implement navigation between views
- Module Development - Create reusable modules

AOT Compatibility

Crystal.Avalonia is fully compatible with .NET AOT (Ahead-of-Time) compilation and trimming.

What is AOT?

AOT compilation converts your .NET code to native code at build time, rather than at runtime via JIT compilation. This results in:

- Faster startup time
- Reduced memory footprint
- Smaller deployment size (with trimming)
- No JIT overhead

Crystal.Avalonia AOT Features

IsAotCompatible

The library is marked as AOT-compatible:

```
<PropertyGroup>
    <IsAotCompatible>true</IsAotCompatible>
</PropertyGroup>
```

DynamicallyAccessedMembers Annotations

All generic methods that use reflection (like `Activator.CreateInstance`) are properly annotated:

```
public static void AddMvvmBindingTransient<
    [DynamicallyAccessedMembers(DynamicallyAccessedMemberTypes.PublicConstructors)] TView,
    [DynamicallyAccessedMembers(DynamicallyAccessedMemberTypes.PublicConstructors)]
TViewModel>
    (this IServiceCollection services)
    where TView : Control where TViewModel : class
{
    // ...
}
```

This tells the trimmer exactly what members are needed at runtime, preventing accidental removal.

Publishing with AOT

Desktop Application

```
dotnet publish -c Release -p:PublishAot=true
```

With Trimming

```
dotnet publish -c Release -p:PublishTrimmed=true -p:PublishAot=true
```

Show Trim Warnings

```
dotnet publish -c Release -p:ShowTrimmedWarnings=true
```

Project Configuration

Example .csproj

```
<Project Sdk="Microsoft.NET.Sdk">

    <PropertyGroup>
        <OutputType>WinExe</OutputType>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <IsAotCompatible>true</IsAotCompatible>
    </PropertyGroup>

</Project>
```

Known Considerations

Avalonia Framework

Avalonia itself uses reflection for XAML loading and data binding. When publishing with trimming:

1. **Enable compiled bindings** - This reduces reflection usage:

```
<AvaloniaUseCompiledBindings>true</AvaloniaUseCompiledBindings>
```

2. **Use x:DataType** - Explicit types help the trimmer:

```
<UserControl xmlns:vm="using:Crystal.Avalonia"
             x:DataType="vm:MainViewModel">
```

```
    <TextBlock Text="{Binding Title}"/>
</UserControl>
```

Troubleshooting

Trimmed Types Missing

If you see warnings about trimmed types:

1. Add `[DynamicallyAccessedMembers]` to generic method parameters
2. Use `Preserve<>` attribute on needed types
3. Add entries to `rd.xml` file for complex scenarios

Runtime Missing Types

If types are missing at runtime:

1. Check that all View/ViewModel types are registered
2. Verify module assemblies are not trimmed away
3. Use `PreserveAll` or `PreserveMember` where needed

Further Reading

- .NET AOT Documentation
- Avalonia AOT Tips